React Performance Optimization Guide

This guide covers the most important performance optimization techniques in React, along with definitions, use cases, and code examples.

1. React.memo

Definition: React.memo is a higher-order component that memoizes a component and prevents re-renders if its props have not changed.

When to Use:
- For pure functional components that render the same output given the same props.
- Useful for expensive UI components.

```
const Child = React.memo(({ value }) => {
  console.log("Child re-rendered");
  return <div>{value}</div>;
});

export default function App() {
  const [count, setCount] = useState(0);
  return (
    <div>
      <button onClick={() => setCount(c => c + 1)}>Increment</button>
      <Child value="Static text" />
    </div>
  );
}
```

2. useCallback

Definition: useCallback returns a memoized version of a callback function that only changes if dependencies change.

When to Use:
- When passing functions to child components wrapped in React.memo.
- Prevents re-creation of function references across renders.

```
const Child = React.memo(({ onClick }) => {
  console.log("Child rendered");
  return <button onClick={onClick}>Click Me</button>;
});

export default function App() {
  const [count, setCount] = useState(0);

  const handleClick = useCallback(() => {
    setCount(c => c + 1);
  }, []);

  return (
    <div>
      <p>Count: {count}</p>
      <Child onClick={handleClick} />
    </div>
  );
}
```

3. useMemo

Definition: useMemo memoizes the result of a computation to avoid recalculating on every render.

When to Use:
- For expensive calculations that don't need to be recomputed every render.
- Prevents unnecessary computations.

```
const ExpensiveCalc = ({ num }) => {
  const result = useMemo(() => {
    console.log("Calculating...");
    return num * 2;
  }, [num]);

  return <div>Result: {result}</div>;
};
```

4. Avoid Anonymous Functions in JSX

Definition: Anonymous functions inside JSX create new references on every render, causing child components to re-render unnecessarily.

■ Bad:

```
<button onClick={() => setCount(count + 1)}>Click</button>
```

■ Good:

```
const handleClick = () => setCount(c => c + 1);
<button onClick={handleClick}>Click</button>
```

5. Debouncing

Definition: Debouncing delays the execution of a function until after a certain period of inactivity.

When to Use:
- For search inputs or API calls triggered by typing.
- Prevents too many API requests.

```
useEffect(() => {
  const timer = setTimeout(() => {
    fetchData(query);
  }, 500);
  return () => clearTimeout(timer);
}, [query]);
```

6. Code Splitting with React.lazy

Definition: React.lazy and Suspense enable code splitting by loading components only when needed.

```
const LazyComponent = React.lazy(() => import("./LazyComponent"));

function App() {
  return (
    <React.Suspense fallback={<div>Loading...</div>}>
      <LazyComponent />
    </React.Suspense>
  );
}
```

7. Virtualization

Definition: Virtualization renders only the visible portion of a large list instead of rendering everything at once.

When to Use:
- For long lists (thousands of items).
- Improves rendering performance.

```
import { FixedSizeList as List } from "react-window";

const Row = ({ index, style }) => <div style={style}>Item {index}</div>;

<List height={200} itemCount={1000} itemSize={35} width={300}>
  {Row}
</List>
```

8. SSR and SSG

Definition: SSR (Server-Side Rendering): HTML is rendered on the server, improving SEO and faster first paint.
SSG (Static Site Generation): Pages are pre-built at build time, improving performance.

When to Use:
- SSR → for dynamic pages (Next.js).
- SSG → for blogs, docs, static content.